

Ex.no:5	DATA VISUALIZATION USING MATPLOTLIB

Aim:

To visualize air quality dataset features and distributions using line charts, bar charts, pie charts, histograms, scatter plots, box plots, and multiple line charts using Matplotlib.

ALGORITHM 1: LINE CHART

Objective:

To visualize the PM2.5 pollution levels across different records.

Steps

1. Load the air quality dataset.
2. Select the PM2.5 column.
3. Consider the first set of records for visualization.
4. Plot the PM2.5 values using a line chart.
5. Add title and axis labels.
6. Display the graph.

Interpretation

The line chart shows how PM2.5 concentration varies across different records and helps identify changes in pollution levels.

Program 1: Line Chart

```
plt.figure(figsize=(10,5))
plt.plot(df["PM2.5"].head(30), marker='o')
plt.title("PM2.5 Pollution Trend")
plt.xlabel("Record Number")
plt.ylabel("PM2.5")
plt.grid(True)
plt.show()
```

ALGORITHM 2: BAR CHART

Objective:

To compare the average PM2.5 pollution levels among different cities.

Steps

1. Select the City and PM2.5 columns.
2. Calculate the average PM2.5 for each city.
3. Create a bar chart using the city names and average PM2.5 values.
4. Add title and axis labels.
5. Display the chart.

Interpretation

The bar chart provides a comparison of average PM2.5 pollution levels between different cities in the dataset.

Program 2: Bar Chart

```
city_pm25 = df.groupby("City")["PM2.5"].mean()
plt.figure(figsize=(10,5))
plt.bar(city_pm25.index, city_pm25.values)
plt.title("Average PM2.5 by City")
plt.xlabel("City")
plt.ylabel("Average PM2.5")
plt.xticks(rotation=45)
plt.show()
```

ALGORITHM 3: PIE CHART

Objective:

To visualize the distribution of air-quality records among different cities.

Steps

1. Select the City column.
2. Count the number of records for each city.
3. Calculate their proportions.
4. Create a pie chart.
5. Display percentage labels.
6. Add a suitable title.

Interpretation

The pie chart shows the percentage contribution of each city to the total number of air-quality records in the dataset.

Program 3: Pie Chart

```
city_count = df["City"].value_counts()
plt.figure(figsize=(8,6))
plt.pie(
    city_count.values,
    labels=city_count.index,
    autopct='%1.1f%%'
)
plt.title("Air Quality Records by City")
plt.show()
```

ALGORITHM 4: HISTOGRAM

Objective:

To visualize the distribution of PM2.5 pollution levels.

Steps

1. Select the PM2.5 column.
2. Remove missing values if present.
3. Divide the PM2.5 values into intervals or bins.
4. Create a histogram using Matplotlib.
5. Add title and axis labels.
6. Display the histogram.

Interpretation

The histogram shows the frequency distribution of PM2.5 values and helps understand the occurrence of different pollution levels.

Program 4: Histogram

```
plt.figure(figsize=(8,5))
plt.hist(df["PM2.5"].dropna(), bins=30)
plt.title("Distribution of PM2.5 Levels")
plt.xlabel("PM2.5")
plt.ylabel("Frequency")
plt.show()
```

ALGORITHM 5: SCATTER PLOT

Objective:

To analyze the relationship between PM2.5 and PM10 pollution levels.

Steps

1. Select PM2.5 as the X-axis variable.
2. Select PM10 as the Y-axis variable.
3. Remove records with missing values.
4. Plot the observations using a scatter plot.
5. Add title and axis labels.
6. Display the graph.

Interpretation

The scatter plot helps identify the relationship between PM2.5 and PM10 concentrations and shows whether both pollutants tend to increase or decrease together.

Program 5: Scatter Plot

```
plot_data = df[["PM2.5", "PM10"]].dropna()
plt.figure(figsize=(8,5))
plt.scatter(
    plot_data["PM2.5"],
    plot_data["PM10"],
    alpha=0.3
)
plt.title("PM2.5 vs PM10")
plt.xlabel("PM2.5")
plt.ylabel("PM10")
plt.show()
```

ALGORITHM 6: BOX PLOT

Objective:

To visualize the distribution and identify possible outliers in PM2.5 pollution levels.

Steps

1. Select the PM2.5 column.
2. Remove missing values if present.
3. Create a box plot using Matplotlib.
4. Add a suitable title and axis label.
5. Display the plot.

Interpretation

The box plot shows the central distribution, spread, median, and possible outliers of PM2.5 pollution values.

Program 6: Box Plot

```
plt.figure(figsize=(7,5))
plt.boxplot(df["PM2.5"].dropna())
plt.title("PM2.5 Box Plot")
plt.ylabel("PM2.5")
plt.show()
```

ALGORITHM 7: MULTIPLE LINE CHART

Objective:

To compare PM2.5 and PM10 pollution levels across different records.

Steps

1. Select PM2.5 and PM10 values from the dataset.
2. Consider a fixed number of records for visualization.
3. Plot PM2.5 values using a line chart.
4. Plot PM10 values using another line on the same graph.
5. Add labels and a legend.
6. Display the graph.

Interpretation

The multiple line chart allows comparison of PM2.5 and PM10 pollution levels across different records and helps identify variations in both pollutants.

Program 7: Multiple Line Chart

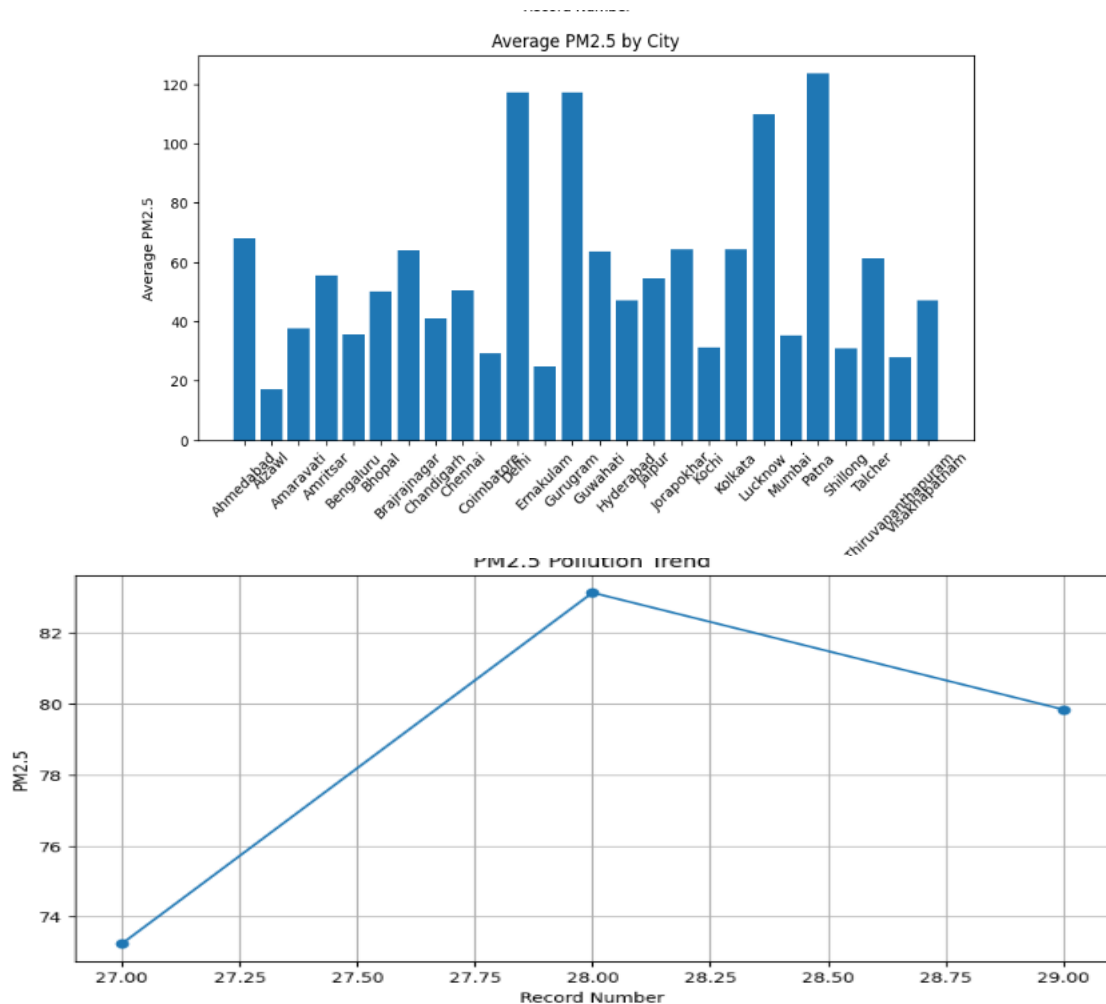
```
data = df[["PM2.5", "PM10"]].head(30)
plt.figure(figsize=(10,5))
lt.plot(
    data["PM2.5"].values,
    marker='o',
    label="PM2.5"
)
plt.plot(
    data["PM10"].values,
```

```

marker='o',
label="PM10"
)
plt.title("PM2.5 and PM10 Pollution Levels")
plt.xlabel("Record Number")
plt.ylabel("Pollution Level")
plt.legend()
plt.grid(True)
plt.show()

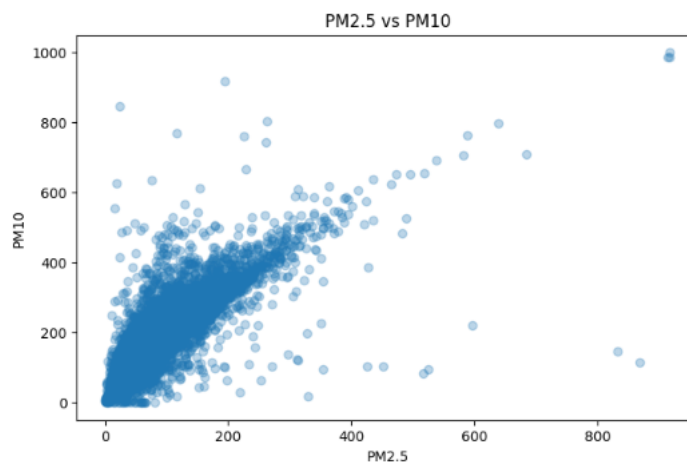
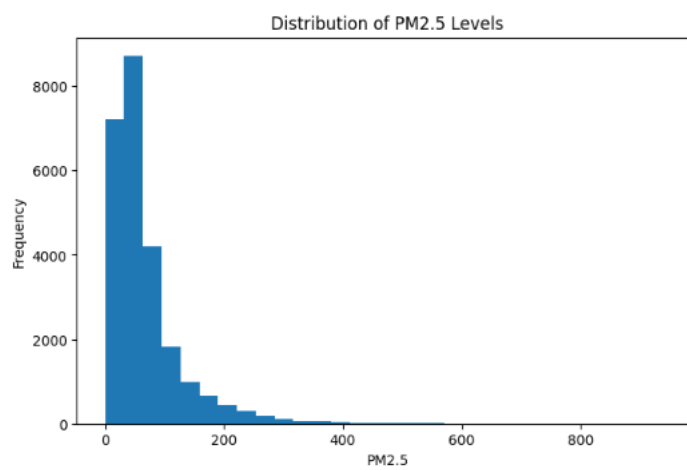
```

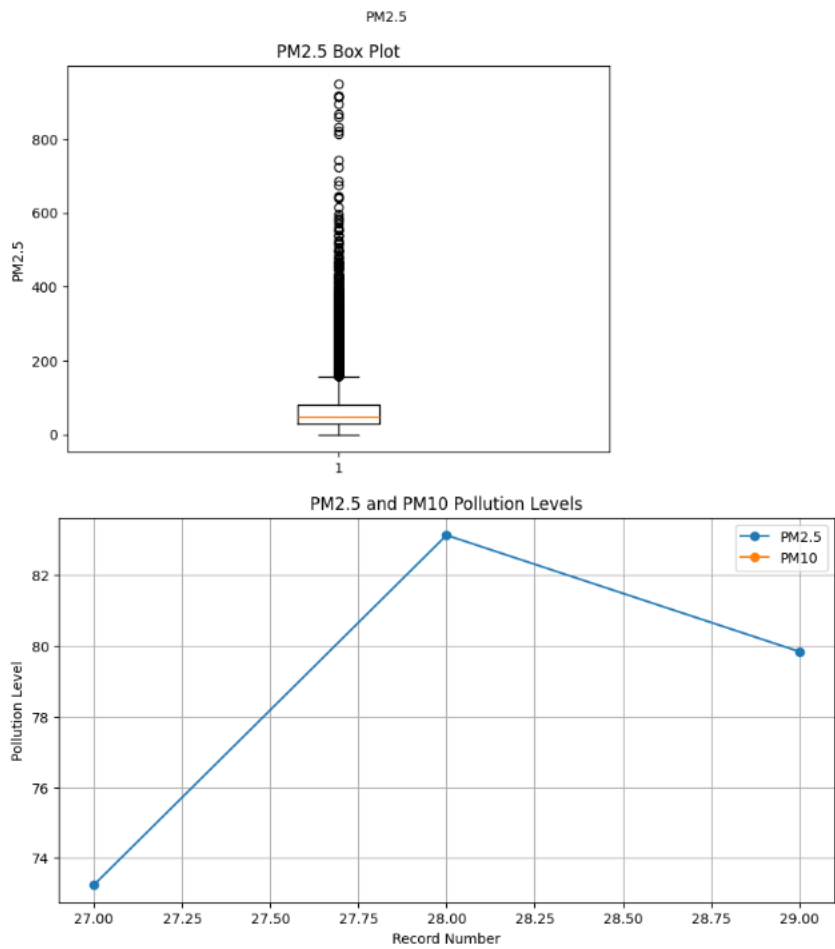
Output:





Distribution of PM2.5 Levels





Result: